



第二章 线性表

杨震

北京邮电大学计算机学院

提纲

BUPT

1

线性表定义

2

向量（顺序表）

3

单链表

4

单循环链表

5

双向链表

6

静态链表

7

线性表的应用

2.1 线性表定义

BUPT

形式定义：由 n ($n \geq 0$) 个数据元素组成的**有序序列**。

$\text{Linear_list} = (D, S)$

其中： $D = \{a_i \mid a_i \in D_0, i=1, \dots, n; n \geq 0\}$

$R = \{N\}$ $N = \{ \langle a_i, a_{i+1} \rangle \mid a_i, a_{i+1} \in D_0, i=1, 2, \dots, n \}$

D_0 为某个数据对象

或者简记为： $(a_1, \dots, a_i, \dots, a_n)$ $n \geq 0$

(n 为**表长**。当 $n=0$ ，称为**空表**)

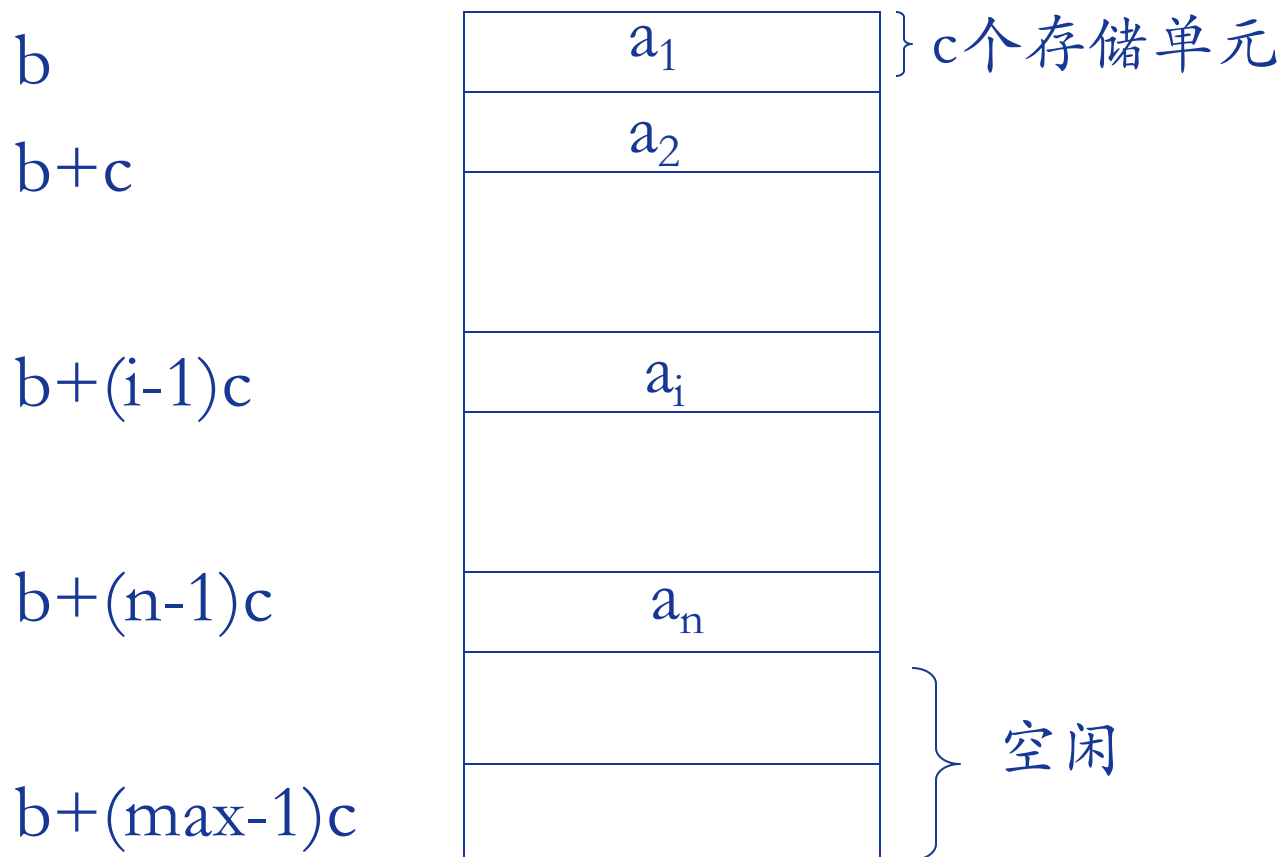
线性表的特点：在数据元素的非空有限集中

- ❖ 数据元素间是线性关系，数据元素在表中的**位置**只取决于其**序号**
- ❖ 存在**唯一**的一个被称作“**第一个**”的数据元素和**唯一**的一个被称作“**最后一个**”的数据元素
- ❖ 除第一个外，每个数据元素均**只有一个**前驱；除最后一个外，每个数据元素均**只有一个**后继

2.2 向量（顺序表）

BUPT

起始地址为 b 、最多可容纳 $\max$ 个元素的向量



顺序表的构造函数与析构函数

BUPT

```
template<typename T>
struct Vector {
    T* elem;\\T为数据元           向量的基地址
    int size;           向量中实际元素个数
    int capacity;       向量能容纳的元素个数
    Vector(int n=0; T const& t=T())
        :elem(0),size(0),capacity(0)
    {
        elem=(T*)malloc(n*sizeof(T));
        uninitialized_fill(elem, elem+n,t);
        size=capacity=n;
    }
    virtual ~Vector() { free(elem);}
}
```

*动态内存分配

❖ malloc 和 free 函数

```
int *p1;... p1=(int *)malloc(25*sizeof(int));...
```

```
int *p2=p1;
```

```
for(i=0;i<25;i++) *p2++=0; 或 p2[i]=0;
```

❖ 常见错误

- 未检查内存是否成功分配;
- 使用 free 是必须是一个从 malloc、calloc/realloc 函数返回的指针，不能释放一块并非动态分配的内存，也不能释放一块动态分配内存的一部分

。

顺序表的判空与求长度

```
template<typename T>
struct Vector {
    bool empty(void) const
    {
        return size==0;
    }
    int size(void) const
    {
        return size;
    }
}
```

顺序表迭代子

BUPT

```
template<typename T>
struct Vector {
    typedef T* Iterator;
    Iterator begin(void) 返回向量首元  
素地址
    {
        return elem;
    }
    Iterator end(void) 返回向量尾元素的下  
一个地址地址
    {
        return elem+size;
    }
}
```


获取顺序表成员

BUPT

```
Template<typename T>
Struct Vector {
    T& front(void)    取向量首元素
    {
        return elem[0];
    }
    T& back(void)    取向量尾元素
    {
        return elem[size-1];
    }
    T& operator[] (int i)    取向量第i个元素
    { // 前提条件:  $0 \leq i \leq \text{size}$ ;
        return elem[i];
    }
}
```

顺序表元素的删除

BUPT

```
Template<typename T>
```

```
Strcut Vector {
```

```
void pop_back(void)
```

```
{
```

```
    size--;
```

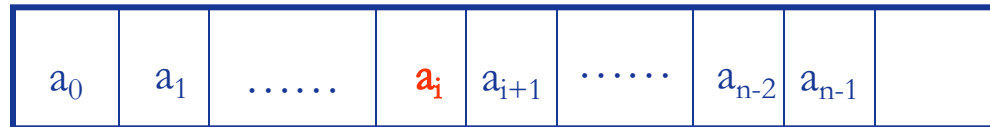
```
}
```

```
void erase(Iterator pos)
```

```
{
```

```
    memmove(pos, pos+1,  
sizeof(T)*(elem+--size-pos));
```

```
}
```



C语言的标准
库函数

```
void* memmove(void* dest, void* source, size_t count)
{
    void* ret = dest;
    if (dest <= source || dest >= (source + count))
    {
        while (count - -)
            *dest++ = *source++;
    }
    else
    {
        dest += count - 1;
        source += count - 1;
        while (count- -)
            *dest-- = *source--;
    }
    return ret;
}
```

```
void strcpy(char *dest, char const *source)
{
    while((*dest++ = *source++) != '\0' );
}
```

对比memmove函数，source指针为const，有什么用？

*source++表达式，他取得source所指向的那个字符，且同时修改source使得它指向下一个字符，但对于形参的修改并不会影响调用程序的实参。

顺序表的存储管理

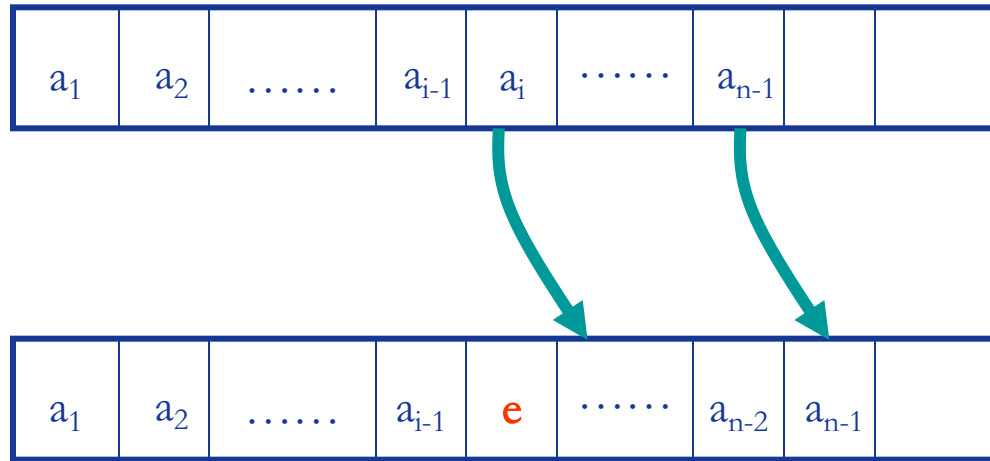
BUPT

```
template<typename T>
Struct Vector {
    void reserve(int n)
    {
        if (n>capacity)
        {
            elem=(T*)realloc(elem,n*sizeof(T));
            uninitialized_fill(elem+size,elem+n,T());
            capacity=n;
        }
    }
protected:
    bool full(void) const
    { return size==capacity; }
    int next_capacity(void) const
    {
        return capacity<= 0 ?1:2*capacity;
    }
}
```

顺序表元素的添加

BUPT

```
template<typename T>
Struct Vector {
    void push_back(T const& t)
    { // 添加在向量尾部
        if (full())
            reverse(next_capacity());
        elem[size++];
    }
    void insert(Iterator pos, T const &t)
    {
        if (full())
        { // 添加在pos之前
            int count offset=pos-elem;
            reserve(next_capacity());
            pos=elem+offset;
        }
        memmove(pos+1,pos,
            sizeof(T)*(elem+size++-pos));
        *pos=t;
    }
}
```



[性能分析]

- 最好 $i=n$ 不用移动结点
时间复杂度 $O(1)$
- 最差 $i=0$ 需移动所有结点
时间复杂度 $O(n)$
- 平均
 $O(n)$

顺序表的其他成员函数

BUPT

```
template<typename T>
void Vector<T>::resize(int n,T const &t=T())
{
    reverse(n);
    if (size<n)
        uninitialized_fill(elem+size,elem+n,T);
    size=n;
}
template<typename T>
void swap(Vector<T>& a, Vector<T>& b)
{
    std::swap(a.elem,b.elem);
    std::swap(a.size,b.size);
    std::swap(a.capacity,b.capacity);
```

❖ 优点

- 逻辑相邻，物理相邻
- 可随机存取任一元素
- 存储空间使用紧凑

❖ 缺点

- 插入、删除操作需要移动大量的元素
- 预先分配空间需按最大空间分配，利用不充分
- 表容量扩充困难

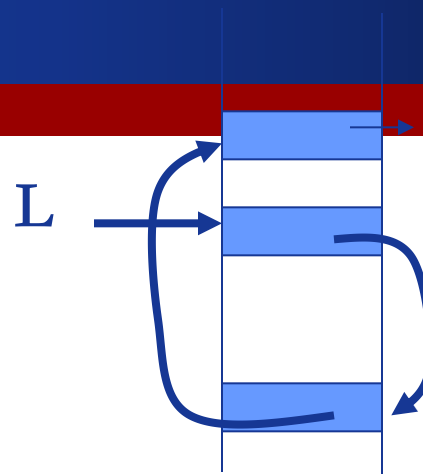
2.3 单链表

BUPT

结点的物理表示



数据域 指针域



两种基本形式

头结点的作用：插入和删除第一个数据元素时不必对头指针进行特殊处理。

头指针L

1264

1264

a_1

a_2

a_n

空表：

$L = \text{NULL}$

头指针

4016

4016

a_1

a_2

a_n

first

头结点

首元结点

带头节点单链表判空：

$L \rightarrow \text{next} \neq \text{NULL}$

单链表节点类型定义

BUPT

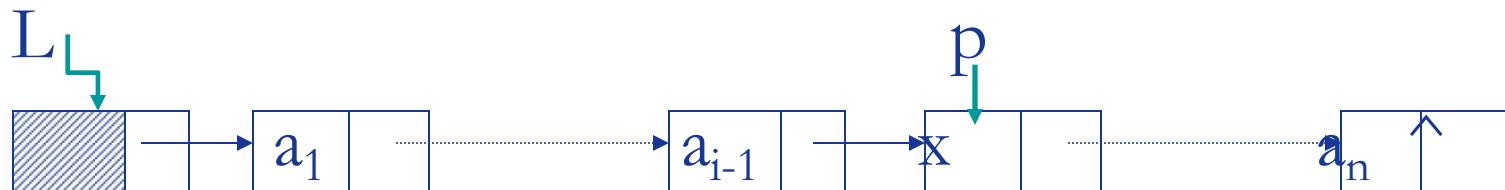
```
struct node
{
    elemtype data;
    struct node *next;
};
```

```
typedef struct node Node;
typedef struct node *PtrToNode;
typedef PtrToNode List;
typedef PtrToNode Position;
```

单链表查找

BUPT

[查找]



Position Find(List L, Element X)

{

 Position p = L->next;

 while (p!=NULL && p->data != X)

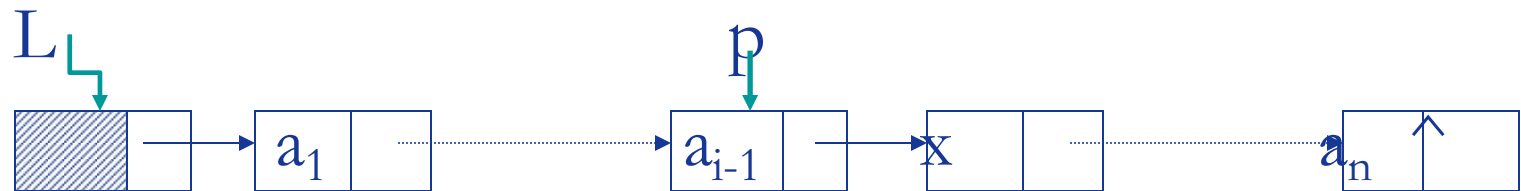
 p=p->next;

 return p;

}

时间复杂度取决于x: 最好 $O(n)$, 最坏 $O(n)$, 等概率下平均 $O(n)$

[查找前驱结点]



Position FindPrevious(List L, elemtype X)

{

 Position p = L;

 while (p->next != NULL && p->next->data != X)

 p=p->next;

 return p;

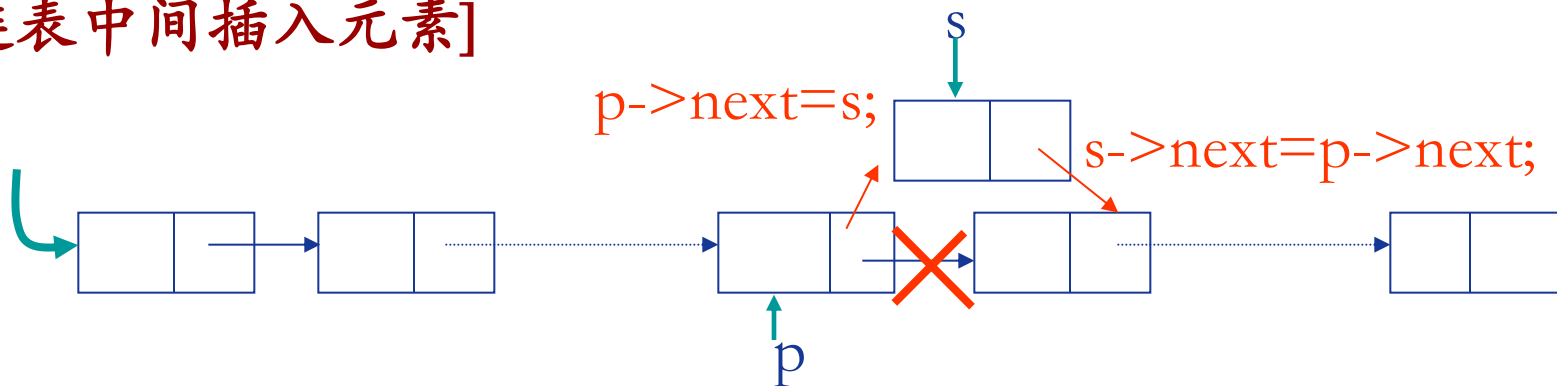
}

时间复杂度取决于x: 最好 $O(n)$, 最坏 $O(n)$, 等概率下平均 $O(n)$

单链表节点的插入

BUPT

[链表中间插入元素]



```
void Insert(List L, Position p, Position s )
```

```
{
```

```
    if(s == NULL || p==NULL)
```

```
        exit( “out of space!” );
```

```
    s->next = p->next;
```

```
    p->next = s;
```

```
}
```

```
s->next = p->next;
```

```
p->next = s; [顺序不能颠倒]
```

[思考]: 在表头插入, 表尾插入情况如何?

头插法与尾插法

BUPT

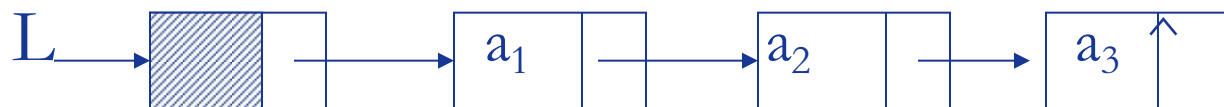
[例1]: 带头节点的单链表的生成

设输入数据元素序列为 $[a_1, a_2, a_3]$

1) 头插法 $O(n)$



2) 尾插法 $O(n^2)$



[思考]: 对于一个具有 n 个结点的单链表, 在已知的结点 (其指针为 p) 后插入一个新结点的时间复杂度为 $O(1)$, 在给定值为 x 的结点后插入一个新结点的平均时间复杂度为 $O(n)$ 。

在单链表已知的结点 (其指针为 p) 前插入一个新结点的时间复杂度又为多少?

有序表中的插入

BUPT

无头结点递增有序单链表的插入

```
BOOL insert(List &L,  elemtype newdata) {  
    Position *; new,current,prev current = L;  
    While(current != NULL && current->data<newdata) {  
        prev= current; current =  current->next;  
    }  
    new =(Node *)malloc(sizeof(Node));  
    判空和赋值语句略...  
    new->next = prev->next;  
    prev->next = new;  
    return TRUE;  
}  
...
```

主函数调用方式:

```
Result = insert(head,X); //head为头指针
```

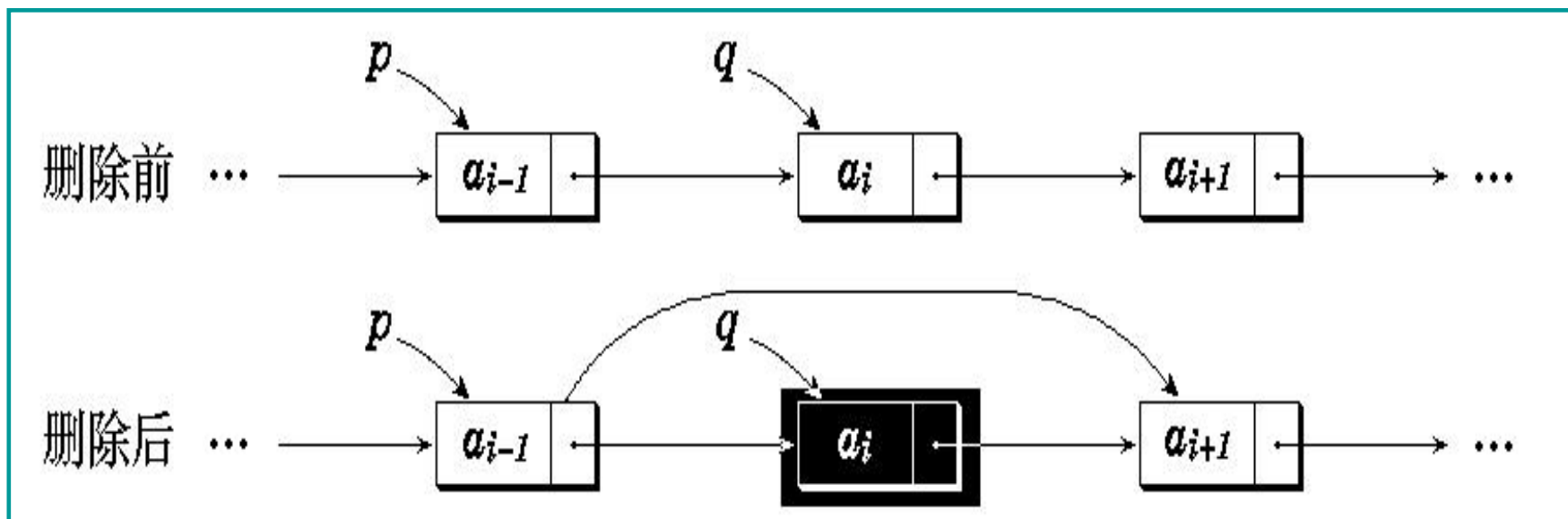

单链表的删除

BUPT

[删除]

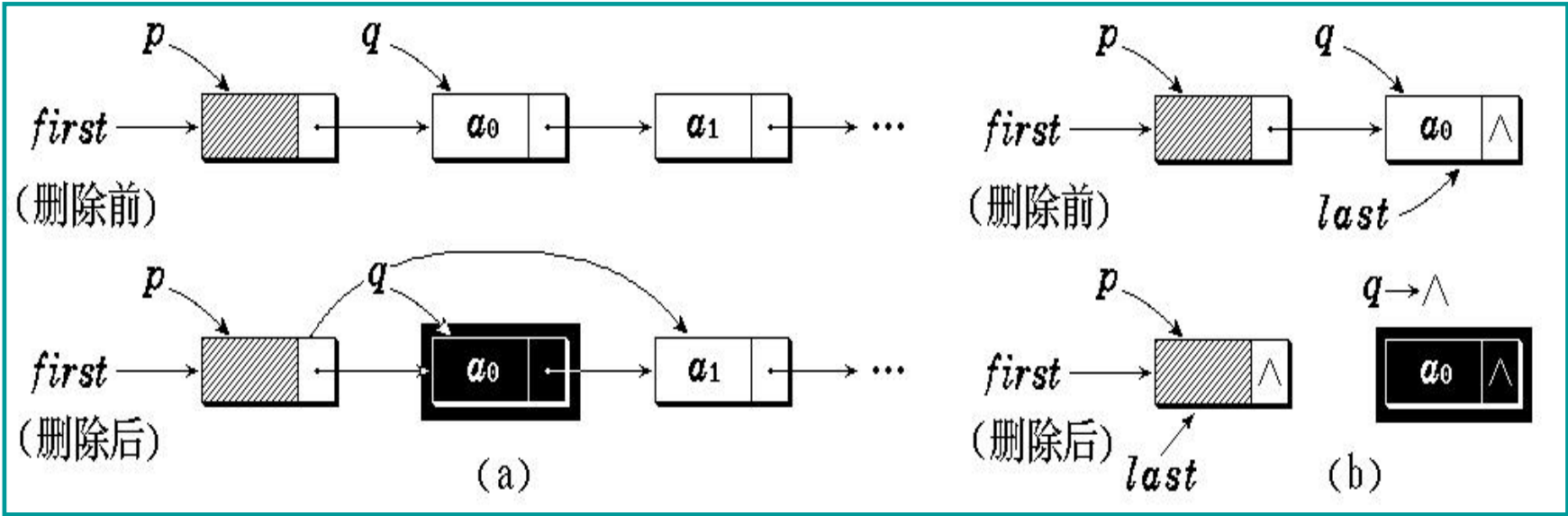
第一种情况: 删除表中第一个元素

第二种情况: 删除表中或表尾元素



```
void DeleteNode( List L ,Position q)//删除表中元素q
{
    p=L;
    while (p->next != NULL && p->next != q)
        p = p->next;
    if(p->next == q)
    {
        p->next = q->next;
        free(q);
    }
}
```

[例2]从带表头结点的单链表中删除第一个结点



单链表操作的综合应用

BUPT

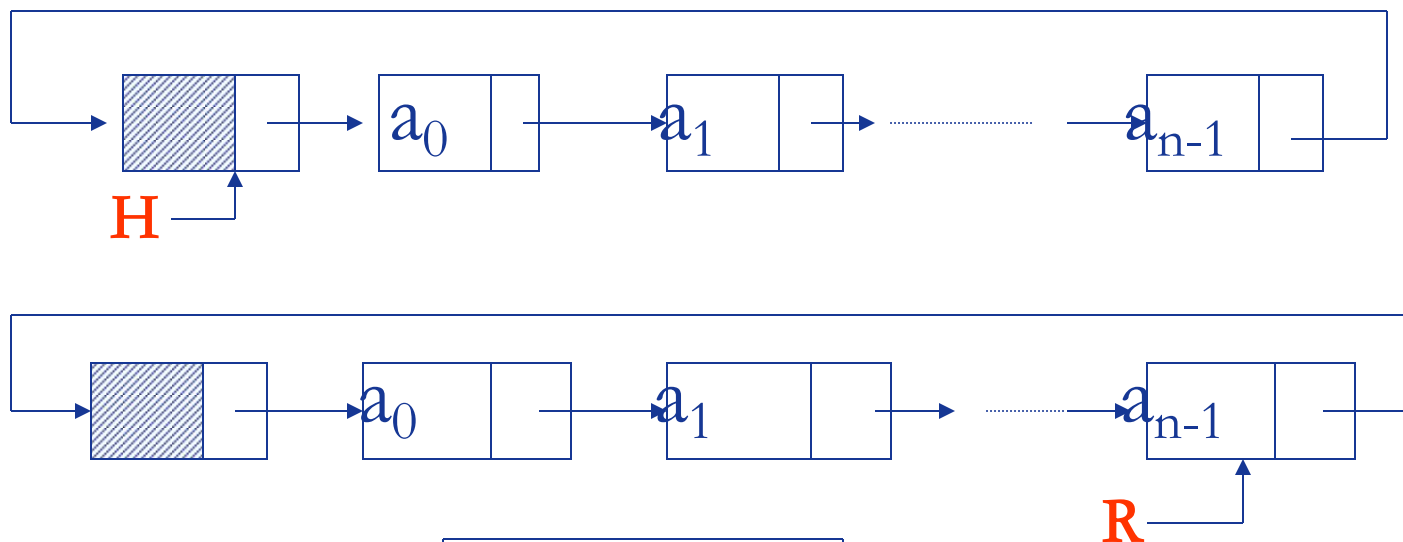
[例3] 以下程序的功能是实现带附加头结点的单链表数据结点逆序连接，请填空完善之。

```
typedef struct node
{int data; struct node *next;
}linknode,*pointer;
void reverse(pointer & h)\\h为头指针
{ pointer p,q;
  p=h->next; h->next=NULL;
  while((1)_____)
  {
    q=p;
    p=p->next;
    q->next=h->next;
    h->next=(2)_____;
  }
}
```

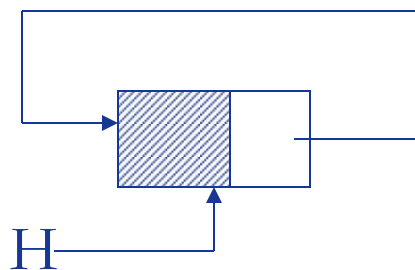
(1)p!=NULL // 链表未到尾就一直作
(2)q // 将当前结点作为头结点
后的第一元素结点插入

2.4 单循环链表

BUPT



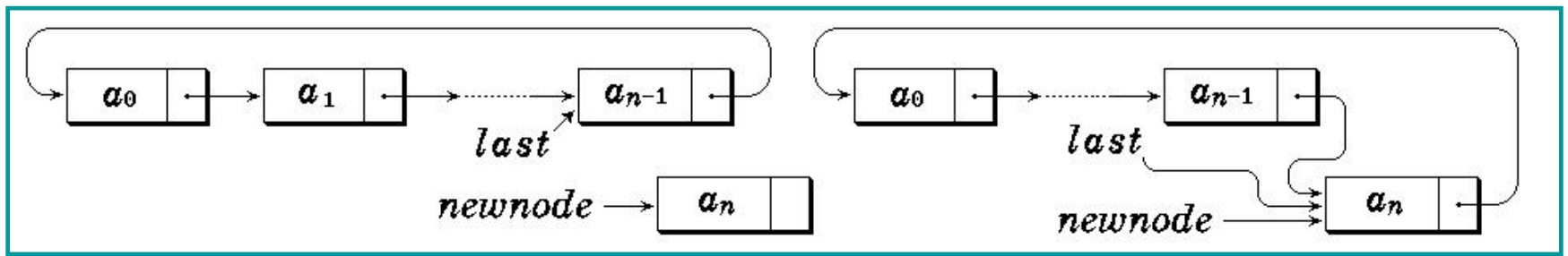
空循环链表



$H \rightarrow \text{next} = H$

循环链表类型定义与单链表相同

❖ 在单循环链表表尾插入一个元素



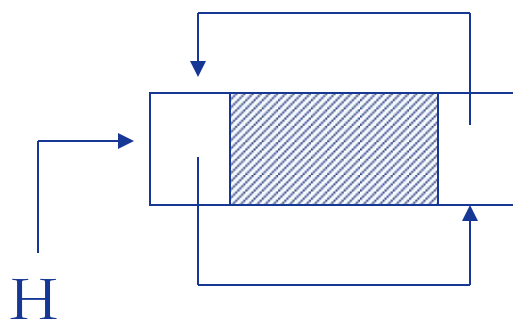
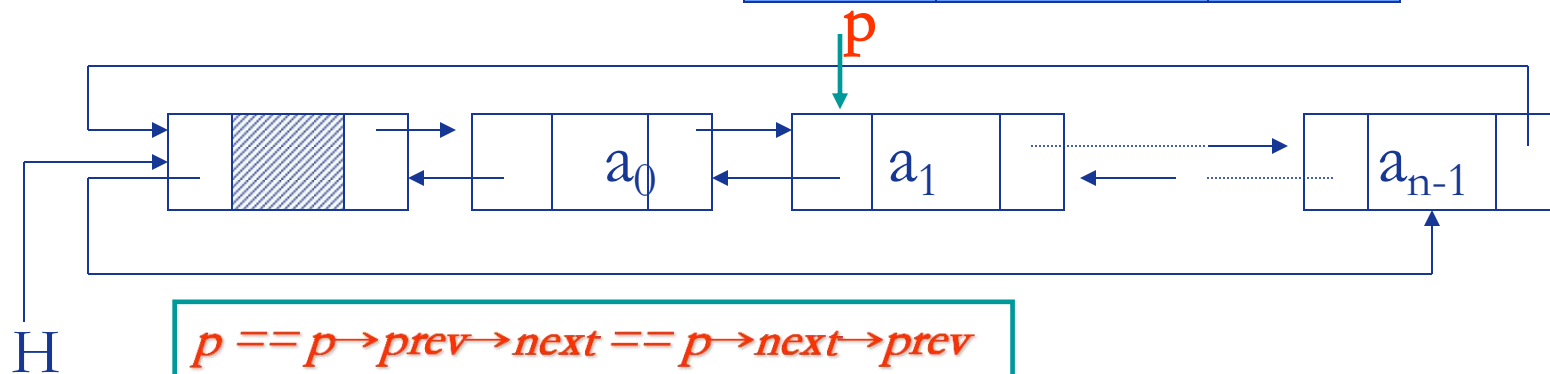
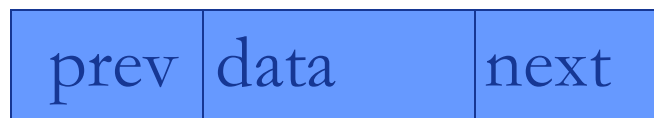
❖ 单循环链表特点

- 是单链表的变形。
- 最后一个结点的后继指针不为 0 (NULL)，而是指向了表的前端。
- 为简化操作，在单循环链表中往往加入表头结点。
- 只要知道表中某一结点的地址，就可搜寻到所有其他结点的地址。

2.5 双向链表

BUPT

结点的物理表示



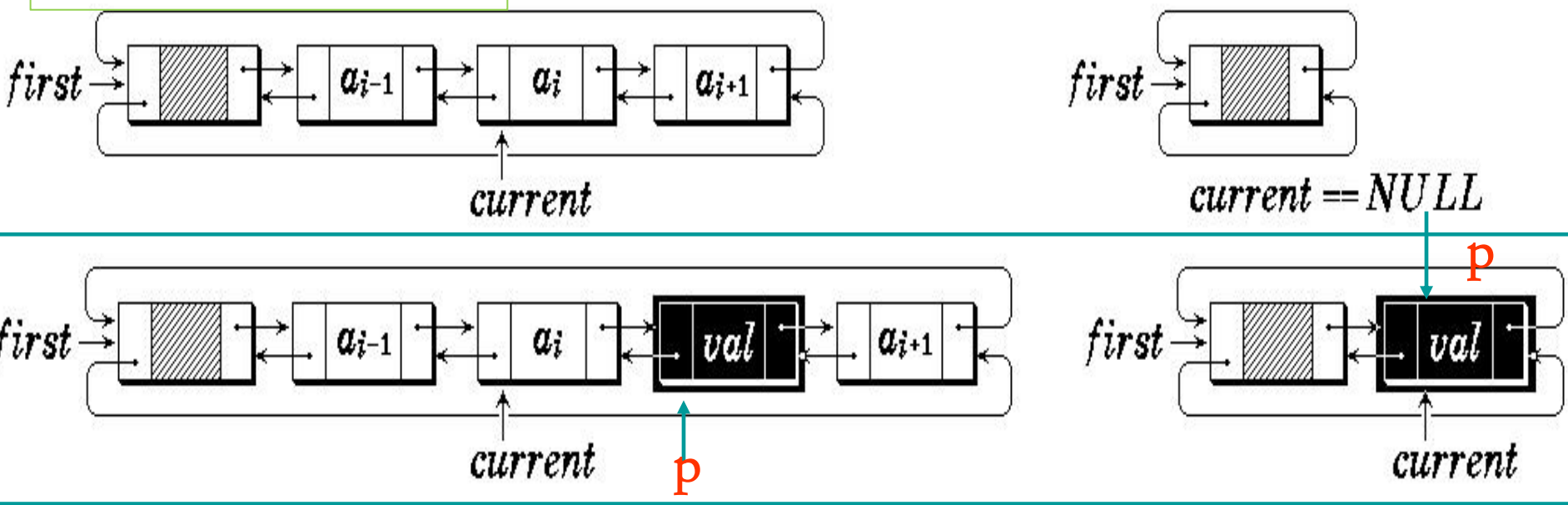
思考：能否以 $H \rightarrow next == H \rightarrow prev$ 判定双向链表为空？

[例]：在current后插入一个节点p，并用current指向插入后的节点

双链表结点定义

```
struct node
{
    elemtype data;
    struct node *next, prev;
};
```

- ① $p \rightarrow prev = current;$
 - ② $p \rightarrow next = current \rightarrow next;$
 - ③ $current \rightarrow next = p;$
 - ④ $current \rightarrow next \rightarrow prev = p;$
- 正确语序是？应该遵循何种原则？



双向循环有序链表的插入算法

```
int insert (Node* head, int value) {  
    Node *thisnode,*nextnode,newnode;  
    for (thisnode=head; (nextnode=thisnode->next)!=head;  
        thisnode =nextnode) {  
        if (nextnode->data==value)      return 0;  
        if (nextnode->data > value)      break;  
    }  
    newnode = (Node*) malloc(sizeof(Node));  
    if (newnode == NULL) return -1 ;  
    nextnode->data= value;  
    newnode->next = nextnode;  
    thisnode->next = newnode;  
    newnode->prev = thisnode;  
    nextnode->prev =newnode;  
    return 1;}
```

顺序表和链表的比较

❖ 空间

- 存储空间静态分配, 需事先确定
- 存储密度高($d=1$) (不考虑空闲区)

❖ 时间

- 是随机存取结构, 可以根据序号直接定位
- 插入/删除结点操作时数据元素要移动

❖ 空间

- 存储空间动态分配, 可以按需使用
- 每一结点附加指针域 ($d<1$)

❖ 时间

- 是非随机存取结构, 定位需从头指针扫描
- 插入/删除结点操作时通常只要修改指针

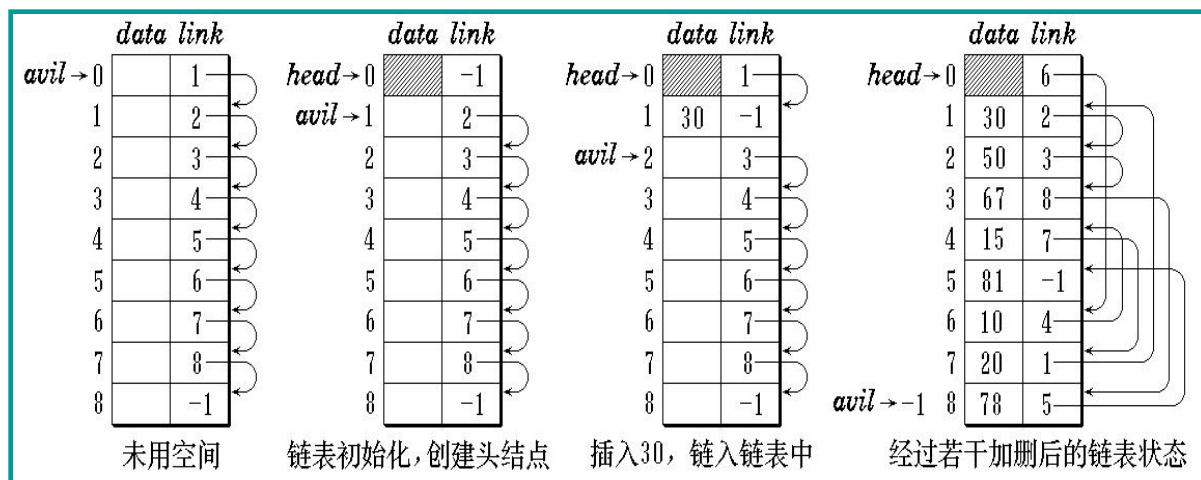
2.6 静态链表

BUPT

❖ 用数组模拟链表—链表的游标实现

- 逻辑上看像链表
- 物理上看是数组

[静态单链表-带头节点]



单链表游标的定义

```
typedef int PtrNode;  
typedef PtrNode List;  
typedef PtrNode Position;
```

```
struct Node
```

```
{  
    elemtype data;  
    Position link;  
}
```

```
Stuct Node A[ApaceSize];
```

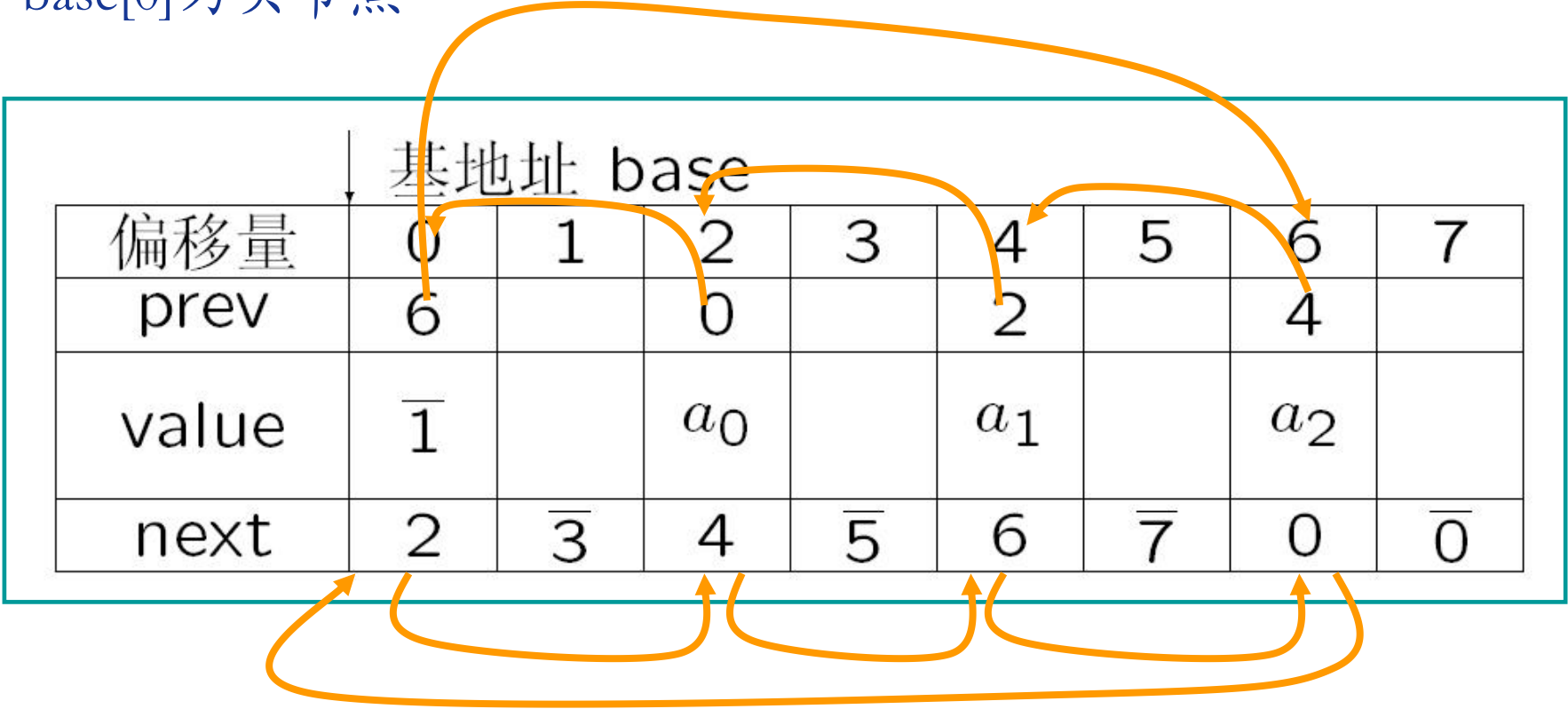
分配节点: $j = avil$, $avil = A[avil].link$;

释放节点: $A[i].link = avil$; $avil = i$;

[静态双链表]

```
struct Stnode { int next, int prev, elemtype data};
```

base[0]为头节点



2.7 线性表应用示例

BUPT

一元n次多项式相加

数学表示

$$p_n(x) = p_0 + p_1x + p_2x^2 + \cdots + p_nx^n$$

$$[\text{例}] \quad p_a(x) = 7 + 3x + 9x^8 + 5x^{17}$$

$$p_b(x) = 8x + 22x^7 - 9x^8$$

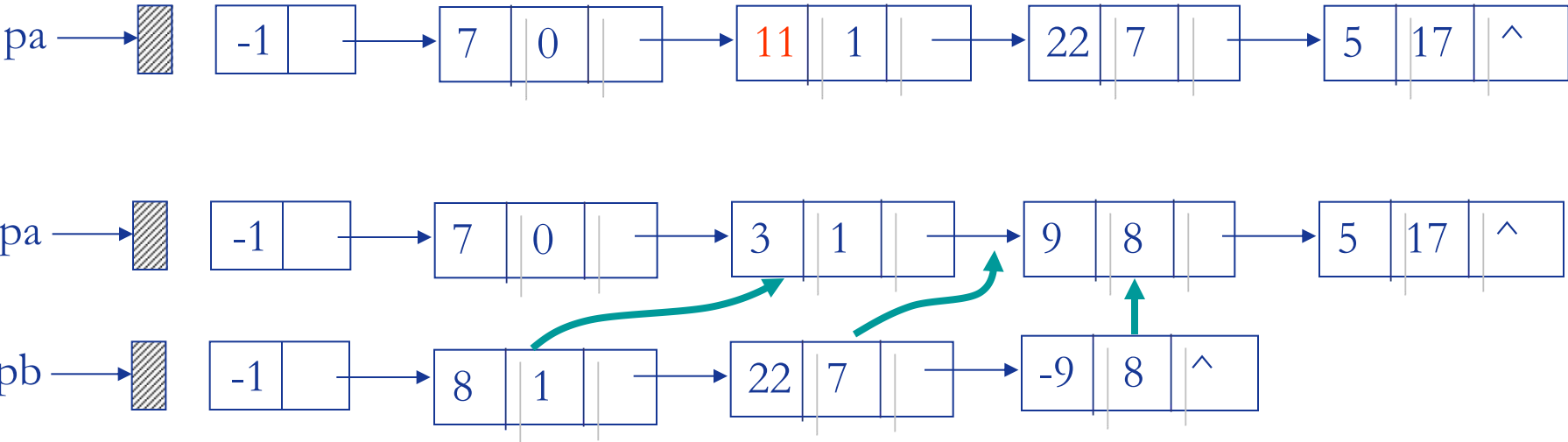
选择数据结构及其存储方式

线性表：数据元素 $\begin{cases} (p) \\ (p, e) \end{cases}$ ✓

存储结构： $\begin{cases} \text{顺序表} \\ \text{带头结点的单链表} \end{cases}$ ✓

存储结构定义

```
typedef struct node{  
double cof;  
int exp; ...  
}
```



```
void add_poly(link & pa, link& pb)
{
    p = pa->next;  q = pb->next;  pre = pa;
    while ((p!=NULL) && (q!= NULL))
    {
        if (p->exp < q->exp)
        if (p->exp == q->exp)
        if ( p->exp > q->exp)
    }
    if (q!=NULL) pre->next=q;
    free(pb);
}
```


❖ 阅读以下算法，填充空格，使其成为完整的算法。其功能是在一个非递减的顺序存储线性表中，删除所有值相等的多余元素。

```
define maxlen=30
Struct node {
    int elem[maxlen];
    int last};
void exam (struct node &L)
{
    j=0;
    i=1;
    while (1)_____
    {
        if (L.elem[i]!=L.elem[j])
        {
            (2)_____;
            (3)_____;
        }
        i=i+1;
    }
    (4)_____;
}
```

(1) $i+1 \leq L.last$ // $L.last$ 为元素个数
 (2) $j++$ // 有值不相等的元素
 (3) $L.elem[j] = L.elem[i]$ // 元素前移
 (4) $L.last = j$ // 元素个数

对于给定的不含头结点的单链表head, 下面的程序过程实现了按结点值非降次序输出单链表中的所有结点, 在每次输出一个结点时, 就把刚输出的结点从链表中删去。请在划线处填上适当的内容, 使之成为一个完整的程序过程, 每个空框只填一个语句。

```
void sort_output_delete (nodeptr head);
{
    nodeptr p,q,r,s;
    while( head !=NULL)
    {   p= NULL ; q= head; r= q ; s=q->next ;
        while ( s != NULL)
        { if (s->data < q->data )
            { (1)___; (2)___ ; }
            r= s ; (3)___;
        }
        printf(q->data) ;
        if (p==NULL)
            (4)___
        else (5)___ ;
        free (q) ;
    }
    .....
}
```

(1)p=r; // r指向指针s的前驱, p指向最小值的前驱。
 (2)q=s; // q指向最小值结点, s是工作指针
 (3)s=s->next // 工作指针后移
 (4)head=head->next; // 第一个结点值最小;
 (5)p->next=q->next; // 跨过被删结点

请写一算法Link LinkListSort(Link list)，将不带头结点的单链表按结点数据域的值的大小从小到大重新链接。要求链接过程中不得使用除该链表以外的任何链结点空间。

```

Link LinkListSort(Link list)
{
    p=list->next; // p是工作指针，指向待排序的当前元素。
    list->next=NULL; // 假定第一个元素有序，即链表中现只有一个结点。
    while(p != NULL)
    {
        r=p->next; // r是p的后继。
        q=list;
        if(q->data>p->data) // 处理待排序结点p比第一个元素结点小的情况。
        {
            p->next=list;
            list=p; // 链表指针指向最小元素。
        }
        else // 查找元素值最小的结点。
        {
            while(q->next!=null && q->next->data<p->data)
                q=q->next;
            p->next=q->next; // 将当前排序结点链入有序链表中。
            q->next=p;
        }
        p=r; // p指向下个待排序结点。
    }
}

```

假设一个单循环链表，其结点含有三个域prev、data、next。其中data为数据域；prev为指针域，它的值为空指针（Null）；next为指针域，它指向后继结点。请设计算法，将此表改成双向循环链表。要求时间复杂度为 $O(n)$

```
void StoDouble (List la)
{
    link p=la;
    while (p->next->prev==NULL)
    {
        p->next->pre=la;    //将结点p后继的prev指针指向p;
        p=p->next;          //la指针后移;
    }
}
```

编写一个算法来交换单链表中指针P所指结点与其后继结点，head是该链表的头指针，P指向该链表中某一结点。

```

LinkedList Exchange (LinkedList head, p)
{
    q=head->next; // q是工作指针，指向链表中当前待处理结点。
    pre=head;     // pre是前驱结点指针，指向q的前驱。
    while (q!=null && q!=p)
    {
        pre=q;
        q=q->next; // 未找到p结点，后移指针
    }

    if (p->next==null)
        printf ( "p无后继结点\n" ); //p是最后一个结点，无后继。
    else //处理p和后继结点交换
    {
        q=p->next; // 暂存p的后继。
        pre->next=q; // p前驱结点的后继指向p的后继。
        p->next=q->next; // p的后继指向原p后继的后继。
        q->next=p ; // 原p后继的后继指针指向p。
    }
}

```

本章作业

BUPT

- ❖ 1. 假设有两个按元素值递增次序排列的线性表，均以单链表形式存储。请编写算法将这两个单链表归并为一个按元素值递减次序排列的单链表，并要求利用原来两个单链表的结点存放归并后的单链表。

节点结构：

```
typedef struct node
{
    int data;
    struct node *next;
} linknode,*link;
```

```
link Union(link la,lb)
```

❖ 2.请在下列算法的横线上填入适当的语句。

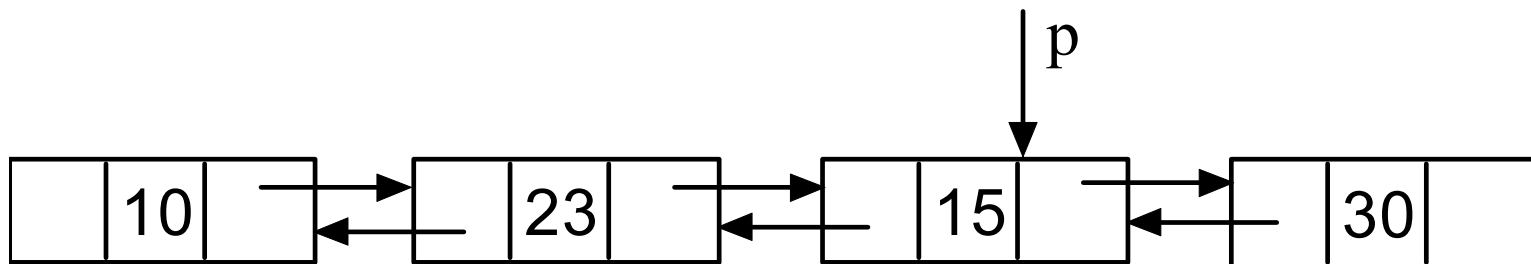
```
typedef struct node
{int data; struct node *next;
}linknode,*link;
bool inclusion(link ha,link hb):boolean;
/*以ha和hb为头指针的带头节点单链表分别表示递增有序表A和B, 本算法判别表
A是否包含在表B内, 若是, 则返回“true”, 否则返回“false” */
{
    pa=ha->next; pb=hb->next; (1);
    while ( (2) )
    {
        if (pa->data==pb->data )
            (3) ;
        else
            (4) ;
    }
    (5) ;
}
```

- ❖ 3. 请写一算法link LinkListSort(link la) , 将不带头结点的单链表按结点数据域的值的大小从小到大重新链接。要求链接过程中不得使用除该链表以外的任何链结点空间。

```
typedef struct node
{int data; struct node *next;
}linknode,*link;
link LinkListSort(link list)
```


❖ 4. 写出下图双链表中对换值为23和15的两个结点相互位置时修改指针的有关语句。

结点结构为：(prev,data,next)



实验报告（一）

❖ 加里森的任务

- 有 n 个加里森敢死队的队员要炸掉敌人的一个军火库，谁都不想去，队长加里森决定用轮回数数的办法来决定哪个战士去执行任务。规则如下：如果前一个战士没完成任务，则要再派一个战士上去。现给每个战士编一个号，大家围坐成一圈，随便从某一个编号为 x 的战士开始计数，当数到 y 时，对应的战士就去执行任务，且此战士不再参加下一轮计数。如果此战士没完成任务，再从下一个战士开始数数，被数到第 y 时，此战士接着去执行任务。以此类推，直到任务完成为止。
- 加里森本人是不愿意去的，假设加里森为1号，请你设计一程序为加里森支招，求出 n, x, y 满足何种条件时，加里森能留到最后而不用去执行任务。
- 要求：
 - 主要数据结构采用链式结构存储。
 - 自拟实验实例验证程序正确性，并讨论 n, x, y 之间的关系。

- 提示：对于 n , x , y 三维度关系讨论可以先固定某个维度值，利用实验数据绘制或讨论其他两维度之间的关系。然后再固定另一维度，重复上述过程。

实验报告撰写格式

****正文抬头部分****

❖ 题目: *****

❖ 班级: ***** 姓名: *** 学号: XXXXXX

1、需求分析

以无歧义的陈述说明所选设计题目的任务，强调的是程序要做什么？明确规定：输入的形式和输出、值的范围；输出的形式；程序所能达到的功能；测试的数据：包括正确的输入和错误的输入及其相应的输出结果；

2、概要设计

问题解决的思路概述；说明程序中用到的所有数据结构类型的定义，主程序的流程以及各程序模块之间的层次（调用）关系。

3、 详细设计

实现概要设计中定义所有数据类型，对每个操作只需要写出伪代码算法，画出函数的调用关系图。

4、 调试分析报告

调试过程中遇到的问题并且是如何解决的以及对设计实现的回顾讨论和分析算法的时空分析（包括基本操作和主要算法的时空复杂度的分析）和改进设想经验和体会等

5、 用户使用说明

向用户说明如何使用你编写的程序，详细列出每一步的操作步骤。

6、 测试结果

详细说明用于测试的实验实例，这里的测试实例应该完整和严格。并列测试出测试结果，包括输入的数据和相应的输出数据。

实验报告提交方式

❖ 提交方式及要求：

- 提交到云平台，源码与实验报告一起打包上传。
- 提交时间：某一章的实验报告应在本章内容结束后两个星期之内提交，逾期不交者不予批改和验收。
- 实验验收：期末验收，时间地点另行通知，要求自带PC和原程序。

附录：AI辅助的软件开发模式

- (1) 根据自我能力选择系统架构（PC或者手机）
- (2) 选用大模型，可选码上大模型、通义灵码、文心一言、星火等）
- (3) 利用大模型进行系统总体设计，并编写需求文档；
- (4) 利用大模型进行算法的对比、选型和设计；
- (5) 利用大模型编写设计文档；
- (6) 利用大模型进行算法的实现；
- (7) 利用爬虫构造数据集，进行测试；
- (8) 利用大模型编写测试文档和用户使用手册；